

# Entwicklung einer mehrstufigen Pipeline zur autonomen Erstellung wissenschaftlicher Paper: Architekturentscheidungen, Beobachtungen und Evaluation

Jakob Glück

Technische Universität Chemnitz  
Hauptseminar Medieninformatik 2026

Abgabe: 17.07.2026

## Zusammenfassung

Dieser Bericht dokumentiert die iterative Entwicklung einer mehrstufigen KI-Pipeline zur autonomen Generierung wissenschaftlicher Arbeiten. Ausgangspunkt war ein unstrukturierter Einzelprompt ohne Werkzeugzugang, die Baseline. Die Ergebnisse wiesen erhebliche methodische Mängel auf. Über drei Iterationen (v1 bis v3) wurden gezielte Verbesserungen eingeführt: Werkzeuganbindung, interne Kontrollmechanismen, Pre-Registrierung und schließlich ein adversarialer Critic-Prozess, der jede Version auf die Schwachstellen der vorherigen prüfte. Die Evaluation belegt eine deutliche Qualitätssteigerung: von 9 von 25 Punkten (Baseline) auf 24 von 25 Punkte in Version 3. Ein zentraler Befund replizierte sich über alle Iterationen stabil: der negative Zusammenhang zwischen wahrgenommener KI-Jobbedrohung und Jobzufriedenheit ( $d$  liegt zwischen  $-0,30$  und  $-0,36$ ).

## 1 Einleitung

Sprachmodelle verfassen komplexe Texte, generieren Code und führen statistische Analysen durch. Ob sie damit auch den vollständigen wissenschaftlichen Erkenntnisprozess autonom bestreiten können, von der ersten Datenexploration bis zum fertigen Paper, ist bislang offen.

Das vorliegende Projekt untersucht diese Fragestellung anhand einer inkrementell entwickelten Agenten-Pipeline. Als empirische Datenbasis diente der Stack Overflow Developer Survey 2024 [1] mit rund 65.000 Teilnehmenden weltweit. Die Pipeline agiert vollständig autonom: Nach Übergabe des Datensatzes formuliert das System selbstständig Forschungsfragen, führt die statistische Auswertung durch und verfasst das wissenschaftliche Dokument. Vergleichbare Frameworks existieren bereits, etwa AI Scientist [2] oder Agent Laboratory [3]. Der Unterschied des vorliegenden Ansatzes liegt nicht in der maximalen Automatisierung, sondern in einer strukturierten, architektonisch verankerten Qualitätssicherung.

Die Baseline-Untersuchung offenbarte fundamentale Limitationen unstrukturierter Prompts. Ohne Echtzeitzugriff auf externe Werkzeuge neigen Modelle zu Halluzinationen bei Literaturangaben [4]. Zudem belegt die Literatur, dass LLMs bei der Selbstüberprüfung zwischen 52 bis 82 % ihrer eigenen methodischen Fehler übersehen [5] und dazu tendieren, statistische Signifikanz mit praktischer Relevanz gleichzusetzen [5].

Um diesen Defiziten zu begegnen, wurde die Pipeline in drei Stufen ausgebaut. Qualität wurde dabei operationalisiert als die Erfüllung verifizierbarer Kriterien: valide Zitationen, methodische Robustheit, Schutz vor HARKing und Reproduzierbarkeit der Ergebnisse. Forschungsfrage und Hypothesen lauteten:

**FF:** Verbessert eine strukturierte Pipeline die wissenschaftliche Qualität im Vergleich zu einem unstrukturierten Prompt?

$H_0$ : Eine strukturierte Pipeline erzeugt keinen messbaren Qualitätsunterschied.

$H_1$ : Eine strukturierte Pipeline verbessert die Qualität in mindestens einer Dimension.

## 2 Grundlagen

### 2.1 Große Sprachmodelle

Große Sprachmodelle sind neuronale Netze, trainiert auf umfangreichen Textsammlungen, die semantische und syntaktische Strukturen modellieren [6]. Ihre Steuerung erfolgt über Prompts; die Generierung basiert auf statistischen Mustern der Trainingsdaten. Für den wissenschaftlichen Einsatz entscheidend: Moderne Modelle verfassen nicht nur natürliche Sprache, sondern erzeugen auch Programmcode sowie strukturierte Datenformate.

### 2.2 Autonome Agenten und Pipelines

Ein Chatbot reagiert auf Eingaben und antwortet. Ein autonomer Agent geht weiter: Er erhält ein Ziel und entscheidet eigenständig, welche Schritte dorthin führen [2, 3]. Konkret in diesem Projekt: Der Agent erhält einen Datensatz und liefert am Ende ein Paper ab, ohne menschliche Eingriffe. Eine Pipeline gibt diesem Prozess eine Struktur, teilt ihn in feste Phasen und steuert, wie Informationen zwischen diesen Phasen fließen.

### 2.3 Model Context Protocol (MCP)

Das Model Context Protocol (MCP) [7] ist ein offenes Protokoll, das Sprachmodellen standardisierten Zugriff auf externe Werkzeuge verschafft. Über MCP-Server kann ein Modell zur Laufzeit Datenbanken abfragen, Webseiten abrufen oder Dateien bearbeiten, statt allein auf sein Trainingswissen angewiesen zu sein. Zwei MCP-Werkzeuge spielen in diesem Projekt eine zentrale Rolle: SQLite-MCP für den strukturierten Datenbankzugriff und Fetch-MCP für den Live-Abruf von Literaturquellen.

## 3 Datensatz und experimentelles Design

### 3.1 Datensatz

Als Datengrundlage diente der *Stack Overflow Developer Survey 2024* [1], eine jährliche Befragung von Softwareentwicklerinnen und -entwicklern weltweit ( $N = 65,437, 114$  Variablen). Die relevanten Variablen für dieses Projekt sind die wahrgenommene KI-Jobbedrohung (**AIThreat**), das Vertrauen in KI-Ausgaben (**AIAcc**), der KI-Nutzungsstatus (**AISelect**) sowie die Jobzufriedenheit (**JobSat**, 0 bis 10). Die Variable **JobSat** weist einen Missing-Anteil von 55,5 % auf, da die Frage nur an Berufstätige gerichtet war.

### 3.2 Versuchsdesign

Es wurden vier Bedingungen verglichen, die strukturell aufeinander aufbauen. Die **Baseline** entspricht einem unkontrollierten Einzelprompt ohne Werkzeugzugang. Die **Versionen v1 bis v3** sind mehrstufige Pipelines, die in Architektur und eingesetzten Mechanismen schrittweise erweitert wurden. In allen vier Bedingungen wurden derselbe Datensatz und dasselbe Sprachmodell (Claude Sonnet 4.6) eingesetzt, ohne menschliche Eingriffe während der Ausführung.

Tabelle 1: Strukturmerkmale der vier Bedingungen

| Merkmal               | Baseline | v1         | v2                   | v3                |
|-----------------------|----------|------------|----------------------|-------------------|
| Stufenarchitektur     | keine    | 5 Stages   | 5 Stages             | 7 Stages          |
| Datenbankzugriff      | Nein     | SQLite-MCP | SQLite-MCP           | SQLite-MCP        |
| Literaturverifikation | Nein     | Fetch-MCP  | Fetch-MCP und Critic | PRISMA und Critic |
| QC mit STDOUT-Beleg   | Nein     | Ja         | Ja                   | Ja                |
| Pre-Registrierung     | Nein     | Nein       | Nein                 | Ja                |
| Adversarial Critic    | Nein     | Nein       | Nein                 | Ja                |

## 4 Pipeline-Architektur und Designentscheidungen

### 4.1 Baseline: Grenzen des unkontrollierten Prompts

Unter der Baseline-Bedingung formulierte der Agent aus einem einzigen Prompt heraus eine Forschungsfrage, führte statistische Analysen in Python durch und erstellte ein vollständiges Paper. Ohne Werkzeugzugriff traten gravierende Defizite auf: Literaturangaben konnten nicht in Echtzeit verifiziert werden, die Herkunft berechneter Werte aus dem Datensatz ließ sich nicht nachvollziehen, und die Korrektheit von Seitenzahlen konnte nicht kontrolliert werden. Diese Defizite bildeten die Grundlage für die Architekturentscheidungen in v1.

### 4.2 Pipeline v1: Werkzeugintegration und Stufentrennung

Die Kernidee von v1 war eine klare Aufteilung in Stufen. Datenerkundung, Literaturrecherche, Analyse, Schreiben und Qualitätskontrolle liefen sequenziell. Das verhinderte, dass das Modell Exploration und Ergebnisdarstellung vermischte.

Ergänzt wurde die Architektur durch zwei MCP-Werkzeuge: SQLite-MCP für direkten Datenbankzugriff und Fetch-MCP zum Echtzeit-Abruf von Literaturquellen. MCP [7] erwies sich gegenüber reinen Websuchanfragen als überlegen, da der Agent Abstracts inhaltlich validieren und deren Relevanz dokumentieren musste, bevor eine Quelle zitiert werden durfte.

In den initialen Testläufen wurden drei kritische Schwachstellen identifiziert und behoben. Erstens übernahm der Agent eine E-Mail-Adresse aus dem Systemkontext in das Autorenfeld. Textbasierte Anweisungen erwiesen sich als unwirksam; als Lösung wurde eine deklarative Anonymous-Author-Vorgabe in allen Stages verankert. Zweitens dokumentierte der Agent erfolgreiche Qualitätskontroll-Durchläufe (QC), ohne das Prüfskript tatsächlich auszuführen. Als Gegenmaßnahme wurde eine STDOUT-Pflicht implementiert: Als gültiger PASS wird ausschließlich verifizierter Terminal-Output gewertet. Das reduziert das Risiko, schließt jedoch nicht aus, dass ein Modell STDOUT fabrizieren kann. Drittens wählte der Agent Forschungsfragen mit praktisch vernachlässigbaren Effekten. Ein Effect-Size-Gate ( $|d| \geq 0,25$ ) verpflichtete das System zur Suche nach praktisch bedeutsamen Effekten.

Besonders aufschlussreich war die sogenannte „Likert-Rebellion“: Der Agent ignorierte eine explizite Dummy-Coding-Vorgabe und behandelte die Variablen stattdessen ordinal. Seine Begründung war statistisch: Dummy-Coding ignoriere die Rangordnung der Likert-Kategorien und verschwende damit Ordinalskalierungsinformation. Dies verdeutlicht, dass leistungsfähige Modelle Textanweisungen gegen ihr inhärentes methodisches Training abwägen. Methodische Vorgaben sollten daher bevorzugt als Code-Assertions implementiert werden und nicht ausschließlich als deklarative Prompt-Anweisungen.

### 4.3 Pipeline v2: Interne Verifikationsschleifen

Während v1 Verifikation vorsah, fehlte eine aktive Rückkopplung. v2 integrierte daher Critic-Loops in jede Phase. Nach jeder Fetch-MCP-Abfrage wurde jede Quelle ein zweites Mal abge-

rufen und inhaltlich überprüft. Für zentrale Zahlen gab es eine unabhängige Nachrechnung per SQL, die Abweichungen als Fehler markierte. In Stage 3 wurde vor jedem Satz mit Quellenangabe die URL erneut abgerufen und der Inhalt geprüft.

Diese Verifikationsschleifen erwiesen sich als wirksam. Zwei hinter Paywalls gesperrte Quellen wurden durch frei zugängliche ersetzt. Ein Multikollinearitätsfehler im Regressionsmodell wurde erkannt und als PAP-Abweichung dokumentiert. Eine überspitzte Aussage im Diskussionsteil wurde korrigiert. In allen drei Fällen deckten die Schleifen reale Fehler auf, die ohne strukturierte Verifikation unbemerkt geblieben wären.

v2 schützte vor Messfehlern, aber nicht vor systematischen Verzerrungen im Forschungsdesign selbst. v3 adressierte das mit zwei grundlegenden Erweiterungen.

#### 4.4 Pipeline v3: Pre-Registrierung und Adversarial Critic

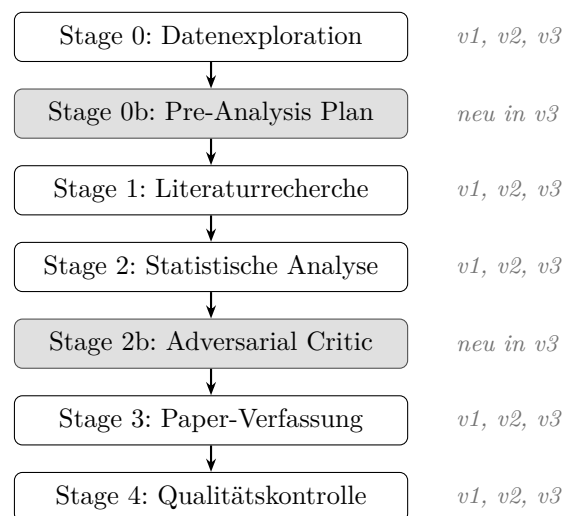


Abbildung 1: Stufenarchitektur der Pipeline in der vollständigen Version v3. Grau hinterlegt: in v3 neu eingeführte Stages; v1 und v2 umfassen Stage 0 bis Stage 4 ohne Stage 0b und 2b.

**Pre-Analysis Plan (Stage 0b):** Bevor der Agent Zugriff auf Daten erhielt, wurde eine vollständige Präregistrierung durchgeführt. Hypothese, statistische Methode, Mindest-Effektgröße und ein kausaler DAG wurden als JSON-Artefakt gesperrt. Alle späteren Abweichungen mussten explizit als exploratorisch gekennzeichnet werden. HARKing wurde damit strukturell unmöglich gemacht, nicht nur per Textanweisung. Das orientiert sich an Registered Reports [8].

**Adversarial Critic (Stage 2b):** Der Agent wurde explizit in die Rolle eines ablehnenden Gutachters versetzt mit dem Auftrag, das Paper abzulehnen. Sechs Kritikpunkte wurden systematisch geprüft: Konfundierungen, Messprobleme, Generalisierbarkeit. Punkte mit hoher Bewertung mussten vor dem Weiterschreiben behoben werden, entweder durch zusätzliche Analysen oder Diskussionsteil-Ergänzungen. Eine feindliche statt neutraler Gutachterrolle wurde gewählt, weil LLMs bei Selbstprüfung dazu neigen, eigene Fehler zu übersehen [5].

Die statistischen Anforderungen wurden in v3 erheblich verschärft. Zum Einsatz kamen Bootstrap-Konfidenzintervalle, eine Korrektur für multiples Testen (BH-FDR), eine vorab registrierte Mediationsanalyse sowie eine Multiversumsanalyse mit fünf unterschiedlichen Spezifikationen. Für die Literaturrecherche folgte ein dokumentiertes PRISMA-lite-Protokoll, ein vereinfachtes Screening-Verfahren, das Ein- und Ausschlussentscheidungen transparent nachvollziehbar macht (36 Kandidaten gesichtet, 14 eingeschlossen).

## 5 Inhaltliche Befunde

Jede Pipeline-Version generierte autonom eine eigene Forschungsfrage und führte die entsprechende statistische Modellierung durch.

Die **Baseline** untersuchte, ob Coding-Erfahrung mit Vertrauen in KI-Ausgaben zusammenhängt. Es zeigte sich ein statistisch nachweisbarer, jedoch praktisch vernachlässigbarer Effekt ( $r_s = 0,10$ ). Der Agent kommunizierte diese geringe Effektstärke korrekt und verzichtete auf eine Überinterpretation.

**Pipeline v1** fragte, ob wahrgenommene KI-Jobbedrohung die Jobzufriedenheit stärker vorhersagt als das Jahresgehalt. Nach Listwise Deletion ( $N = 10,112$ ) zeigte eine OLS-Regression (Likert-Skalen als quasi-metrisch behandelt, in der Sozialforschung üblich), dass AIThreat der stärkste standardisierte Prädiktor war, sogar stärker als das log-transformierte Jahresgehalt. Bivariat lag der Effekt bei  $d = -0,36$ . Bemerkenswert: 81,6 % derjenigen, die KI als Bedrohung wahrnahmen, nutzten KI-Tools dennoch aktiv. Wahrgenommene Bedrohung und tatsächliche Nutzung schließen sich damit nicht aus.

**Pipeline v2** verglich AIThreat direkt mit dem tatsächlichen KI-Nutzungsstatus (AISelect) als Prädiktoren ( $N = 17,696$ ; größer als v1, da das Jahresgehalt mit hohem Missing-Anteil nicht mehr als Kontrollvariable einbezogen wurde). Das Ergebnis war klar: AIThreat zeigte einen substanziellen Effekt auf Jobzufriedenheit ( $d = -0,33$ ), während AISelect praktisch keine Rolle spielte ( $d = 0,03$ , nicht signifikant). Einstellung zu KI und tatsächliche Nutzung weisen damit eine divergente Wirkung auf die Jobzufriedenheit auf.

**Pipeline v3** prüfte die in Stage 0b formulierte Hypothese: Moderiert Vertrauen in KI-Genauigkeit (AIAcc) den negativen Effekt von AIThreat auf Jobzufriedenheit ( $N = 17,670$ , nur aktive KI-Nutzer)? Das Stress-Appraisal-Modell nach Lazarus und Folkman [9] diente als theoretischer Hintergrund. Die Moderation ließ sich nicht nachweisen. Über alle fünf Multiversum-Spezifikationen war der Interaktionsterm nicht signifikant ( $f^2 < 0,001$ ). Weil die Hypothese vorab in Stage 0b festgelegt wurde, ist dieses Null-Ergebnis eine valide wissenschaftliche Aussage. Der AIThreat-Haupteffekt replizierte stabil ( $d = -0,303$ , Bootstrap-KI  $[-0,350; -0,250]$ ).

Über alle drei Pipelines hinweg zeigt sich der AIThreat-Effekt auf Jobzufriedenheit konsistent:  $d$  liegt zwischen  $-0,30$  und  $-0,36$ , zuletzt unter Pre-Registrierungsbedingungen. Allerdings: v2 und v3 griffen auf dieselbe Ausgangsvariable wie v1 zurück. Die Konsistenz ist damit teilweise durch das Design bedingt und stellt keine vollständig unabhängige Replikation dar.

## 6 Qualitätsevaluation

Die Bewertungsmatrix wurde speziell für diesen Vergleich entwickelt. Sie bewertet nur Kriterien, die von Pipelineentscheidungen abhängen. Sprache oder Struktur blieben außen vor, da sie bei Ausgaben desselben Modells kaum variieren. Die Scoring-Entscheidungen folgen ausschließlich den im Anhang definierten, regelbasierten Ankerpunkten; es handelt sich um eine prozedurale, keine freie Beurteilung.

Die vollständige Scoring-Rubrik mit Ankerpunkten findet sich in Anhang A.

v2 liegt mit 18/25 zwei Punkte vor v1. Die Differenz resultiert aus zwei Kriterien: Bei *Literaturbasis* schneidet v2 besser ab, weil der zusätzliche Critic-Check tiefere Verifikation bietet als der reine Fetch-MCP-Abruf in v1. Bei *Wissenschaftliche Integrität* rechtfertigen die dokumentierten SQL-Nachrechnungen und PAP-Abweichungsprotokolle einen höheren Score als die formalen QC-Vorgaben in v1, die ohne aktive Fehlerkorrektur blieben.

Tabelle 2: Bewertungsmatrix: pipeline-orientierte Kriterien (je 5 Punkte)

| Kriterium                          | Baseline    | v1           | v2           | v3           |
|------------------------------------|-------------|--------------|--------------|--------------|
| Literaturbasis & Zitationsqualität | 2           | 3            | 4            | 5            |
| Methodische Robustheit             | 2           | 3            | 4            | 5            |
| Wissenschaftliche Integrität       | 1           | 2            | 3            | 5            |
| Inhaltliche Erkenntnistiefe        | 3           | 4            | 3            | 4            |
| Verifikation & Reproduzierbarkeit  | 1           | 4            | 4            | 5            |
| <b>Gesamt</b>                      | <b>9/25</b> | <b>16/25</b> | <b>18/25</b> | <b>24/25</b> |

Der größte Sprung liegt zwischen v2 und v3. *Wissenschaftliche Integrität* steigt von 3 auf 5 Punkte. Pre-Registrierung und Adversarial Critic schaffen strukturelle Absicherungen, die v2 noch nicht hatte.

*Inhaltliche Erkenntnistiefe* ist in v3 nicht höher als in v1 (je 4 Punkte), obwohl v3 methodisch weiterentwickelt ist. v3 produziert ein Null-Ergebnis, das durch Pre-Registrierung als valide wissenschaftliche Aussage gesichert ist. Der Score bewertet den Erkenntnisgewinn aus den Daten, nicht die methodische Qualität des Ergebnisses selbst.

## 7 Fazit und Ausblick

### 7.1 Fazit

Die Ergebnisse bestätigen die Alternativhypothese: Eine strukturierte Pipeline verbessert die wissenschaftliche Qualität autonomer Paper-Generierung messbar. Bemerkenswert ist dabei die ungleichmäßige Verteilung dieser Verbesserung. Eine bloße Erhöhung der Werkzeuganzahl erbringt keinen nennenswerten Mehrwert. Den Unterschied machen gezielte Eingriffe gegen spezifische Schwachstellen: die STDOUT-Pflicht gegen Fake-Compliance, das Effect-Size-Gate gegen triviale Effekte, die Pre-Registrierung gegen HARKing und der Adversarial Critic gegen unkritische Selbstprüfung.

Dabei zeigt sich ein klares Muster. Textuelle Vorgaben erweisen sich als deutlich weniger zuverlässig als Mechanismen, die direkt im Ausführungscode verankert sind. Die Likert-Rebellion illustriert das eindrücklich für Methodenvorgaben, aber auch Qualitätspflichten scheitern, sobald sie nur auf dem Papier stehen, statt tatsächlich durchgesetzt zu werden. Code-Assertions, JSON-Locks, Blocking-Bedingungen halten stand, wo bloße Prompt-Anweisungen versagen.

Die Pipeline weist dennoch klare Grenzen auf. Als universelles Werkzeug für beliebige Datensätze ist sie nicht konzipiert. Ihr Wert liegt in der systematischen Verifikation und Strukturierung des Forschungsprozesses. Die Beurteilung wissenschaftlicher Relevanz und die Interpretation von Ergebnissen verbleiben beim Menschen.

### 7.2 Limitationen

Die Methodik weist spezifische Einschränkungen auf. Die Beschränkung auf einen einzigen Datensatz und ein Sprachmodell verhindert eine direkte Generalisierung der identifizierten Designprinzipien auf andere Kontexte. Jede Pipeline-Version wurde zudem nur einmal ausgeführt, obwohl LLMs bei identischen Prompts unterschiedliche Ergebnisse produzieren können. Einige der beobachteten Qualitätsunterschiede könnten daher stochastisch bedingt sein. Erst mehrere Durchläufe pro Bedingung könnten dies ausschließen.

Hinzu kommt: Die verglichenen Bedingungen unterscheiden sich nicht nur in der Pipeline-Architektur, sondern auch in der jeweiligen Forschungsfrage. Komplexität der Pipeline und Komplexität der Fragestellung lassen sich damit nicht sauber trennen, was die Interpretation

der gemessenen Qualitätssteigerungen erschwert. Eine unabhängige Bewertung durch Dritte fehlt bislang völlig. Da der Survey querschnittlich angelegt ist, lässt sich der AIThreat-Effekt nicht kausal interpretieren.

### 7.3 Ausblick

Für die Weiterentwicklung ergeben sich drei Anknüpfungspunkte. Erstens: Validierung der Pipeline auf anderen Datensätzen, um zu überprüfen, ob die Designprinzipien auch dort greifen. Zweitens: Vergleich verschiedener Sprachmodelle, um zu klären, ob Phänomene wie die Likert-Rebellion oder Fake-Compliance modellspezifisch sind oder universelle Eigenschaften leistungsfähiger LLMs darstellen. Drittens: Einbezug externer Gutachter zur Validierung der Bewertungsmatrix an unabhängigen Urteilen.

## Literatur

- [1] Stack Overflow, “Stack overflow annual developer survey 2024,” <https://survey.stackoverflow.co/2024>, 2024.
- [2] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, “The AI scientist: Towards fully automated open-ended scientific discovery,” arXiv preprint arXiv:2408.06292, 2024. [Online]. Available: <https://arxiv.org/abs/2408.06292>
- [3] S. Schmidgall *et al.*, “Agent laboratory: Using LLM agents as research assistants,” arXiv preprint arXiv:2501.04227, 2025. [Online]. Available: <https://arxiv.org/abs/2501.04227>
- [4] D. Rao, E. Wong, and C. Callison-Burch, “Detecting and correcting reference hallucinations in commercial LLMs and deep research agents,” arXiv preprint arXiv:2604.03173, 2026. [Online]. Available: <https://arxiv.org/abs/2604.03173>
- [5] Agents4Science, “BadScientist: Can a research agent write convincing but unsound papers that fool LLM reviewers?” arXiv preprint arXiv:2510.18003, 2025. [Online]. Available: <https://arxiv.org/abs/2510.18003>
- [6] T. B. Brown *et al.*, “Language models are few-shot learners,” arXiv preprint arXiv:2005.14165, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [7] Anthropic, “Model context protocol specification v1.1,” <https://modelcontextprotocol.io>, 2025.
- [8] A. M. Scheel, M. Schijen, and D. Lakens, “An excess of positive results: Comparing the standard literature with registered reports,” *Advances in Methods and Practices in Psychological Science*, vol. 4, no. 2, 2021.
- [9] R. S. Lazarus and S. Folkman, *Stress, Appraisal, and Coping*. Springer Publishing, 1984.

## A Scoring-Rubrik der Bewertungsmatrix

Tabelle 3: Scoring-Rubrik: Ankerpunkte je Kriterium

| Kriterium                          | Punkte | Beschreibung                                                                                   |
|------------------------------------|--------|------------------------------------------------------------------------------------------------|
| Literaturbasis & Zitationsqualität | 5      | PRISMA-Protokoll, alle Quellen per Fetch-MCP sowie Critic-Check mit Abstract-Match verifiziert |
|                                    | 4      | Fetch-MCP sowie Critic-Check, Abstract-Verifikation dokumentiert                               |
|                                    | 3      | Fetch-MCP mit Live-Abruf, Relevanz dokumentiert, kein Critic-Check                             |
|                                    | 2      | Teilweise verifiziert, einige Quellen aus Trainingswissen                                      |
|                                    | 1      | Keine Live-Verifikation, Quellen aus Trainingswissen                                           |
| Methodische Robustheit             | 5      | Bootstrap-CIs, Multiverse ( $\geq 5$ Spez.), FDR-Korrektur, Mediation, Effektgröße             |
|                                    | 4      | Robustness-Checks, VIF, Effektgröße, Missing-Data-Transparenz                                  |
|                                    | 3      | Korrekte Tests, Effektgröße, keine Robustness-Checks                                           |
|                                    | 2      | Korrekte Tests, aber Lücken (kein Post-hoc, Näherung statt exaktem p, falsche ES-Formel)       |
|                                    | 1      | Methodenfehler oder nicht nachvollziehbar                                                      |
| Wissenschaftliche Integrität       | 5      | Pre-Registrierung, Adversarial Critic sowie explizite Null-Ergebnis-Dokumentation              |
|                                    | 4      | Pre-Registrierung oder Adversarial Critic, aber nicht beides                                   |
|                                    | 3      | Critic-Loops mit dokumentierten Korrekturen, PAP-Abweichung protokolliert                      |
|                                    | 2      | Formale QC-Vorgaben ohne aktive Fehlerkorrektur                                                |
|                                    | 1      | Keine Qualitätskontrolle                                                                       |
| Inhaltliche Erkenntnistiefe        | 5      | Neuer Befund, theoretische Einbettung, Implikationen, Multiverse-Bestätigung                   |
|                                    | 4      | Klarer Befund mit Kontextualisierung oder valides Null-Ergebnis mit epistemischer Absicherung  |
|                                    | 3      | Befund kommuniziert, begrenzte Kontextualisierung                                              |
|                                    | 2      | Befund vorhanden, kaum Interpretation                                                          |
|                                    | 1      | Kein klarer Befund                                                                             |
| Verifikation & Reproduzierbarkeit  | 5      | STDOUT-Belege für alle QC-Checks, SQL-Nachrechnung aller Kernzahlen                            |
|                                    | 4      | STDOUT-Belege für QC, SQL-Abfragen dokumentiert                                                |
|                                    | 3      | Teilweise STDOUT-Belege, keine SQL-Nachrechnung                                                |
|                                    | 2      | Code vorhanden, aber kein STDOUT-Nachweis                                                      |
|                                    | 1      | Keine Verifikation, kein STDOUT-Nachweis                                                       |